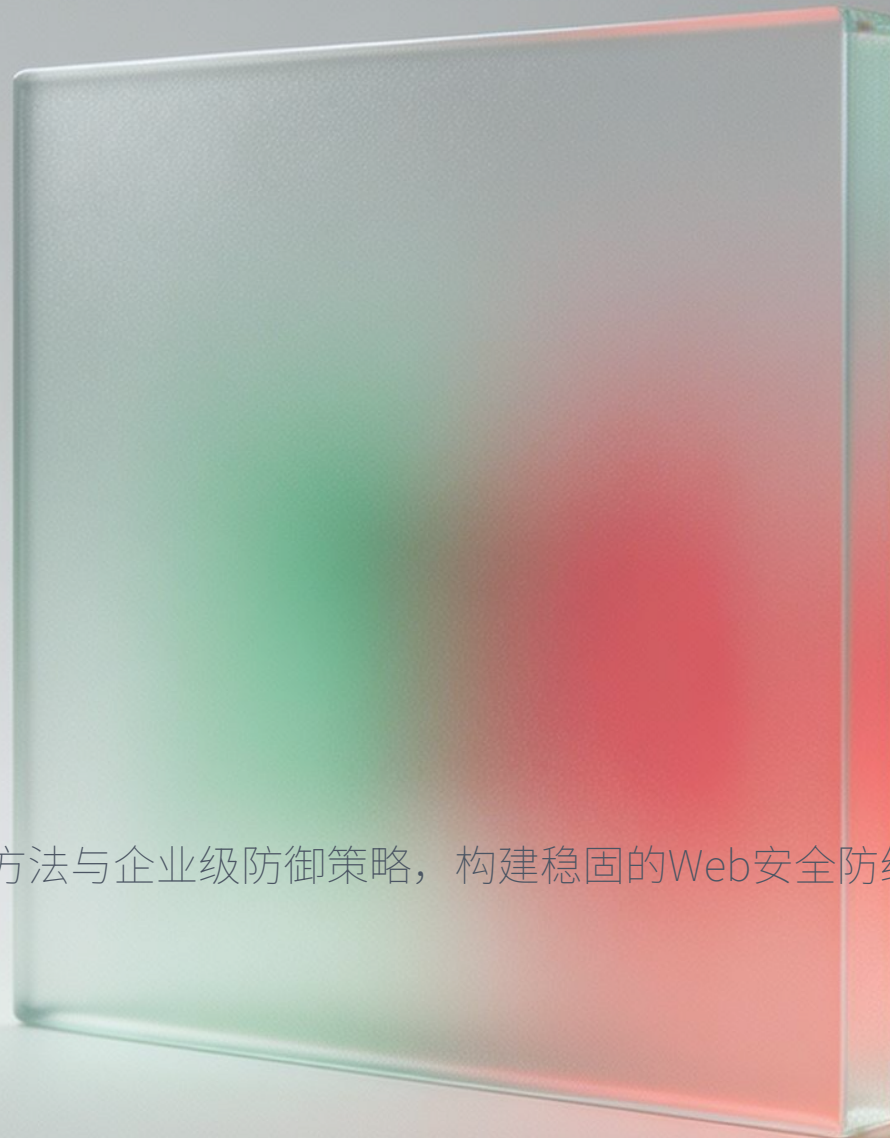


CSRF漏洞讲解

看懂跨站请求伪造

深入解析CSRF漏洞的底层原理、潜在安全危害、检测方法与企业级防御策略，构建稳固的Web安全防线。

汇报人：磊



目录

01

什么是 CSRF

深入解析跨站请求伪造的基础概念，用通俗语言还原攻击原理与触发条件，建立对漏洞的基础认知。

02

CSRF 攻击真实场景举例

通过银行转账、社交账号篡改等贴近生活的真实案例，直观感受攻击是如何在用户毫无察觉下发生的。

03

CSRF 漏洞带来的危害

剖析漏洞对用户隐私泄露、资金安全的威胁，以及对企业数据资产、业务运营稳定性的深远影响。

04

如何检测 CSRF 漏洞

掌握手动检测的核心逻辑与关键步骤，了解 Burp Suite 等主流安全工具在漏洞自动化排查中的应用。

05

CSRF 漏洞的防范手段

详解同源检测、Anti-CSRF Token 验证、双重 Cookie 验证等核心防护策略，构建纵深防御体系。

06

总结与未来安全趋势

回顾本次分享的核心知识点，探讨 Web 安全防护的演进方向，并给出持续学习与实践的建议。

01 什么是CSRF

Cross-Site Request Forgery

中文译名：跨站请求伪造，是一种利用用户在已登录状态下的身份凭证，发起非用户本意请求的攻击方式。

// () 核心原理

攻击者诱导用户点击恶意链接，利用用户已登录的会话凭证（如Cookie），冒充用户身份向目标服务器发起非自愿的请求。

攻击特点

无需窃取用户密码，仅利用浏览器的自动认证机制；攻击过程对用户透明，难以察觉；通常结合钓鱼手段进行诱导。

潜在危害 ()

攻击者可冒充用户执行敏感操作，如修改账号信息、转账汇款、发布恶意内容、删除重要数据等，造成财产损失与隐私泄露。

 通俗理解：攻击者拿着你的钥匙（身份凭证），去你家（网站）做了你不想做的事，而你却毫不知情。

CSRF的定义 (通俗版)



浏览器保管着你的
“数字身份证” (登录凭证)



场景还原：你登录支付宝后，浏览器如同拿着你的“身份证”。此时若访问恶意网站，它会暗中操控你的浏览器，凭这份“身份证”向支付宝发送转账、改密等请求。整个过程，攻击者从未接触你的账号密码。



核心真相：攻击者无法窃取你的账号密码，而是“借壳办事”——盗用你已登录的身份，冒充你执行操作。

✗ 非“偷密码”攻击

无需获取账号密码，利用用户的“登录态”发起请求，受害者往往在毫不知情的情况下完成操作。

✗ 区别于SQL注入

SQL注入是直接攻击服务器数据库，窃取数据；而CSRF是劫持用户浏览器，冒充用户身份进行操作。

CSRF完整工作原理（两步拆解）

⚠ **攻击前提：** 用户必须处于登录目标网站的状态（Cookie有效且未过期），这是攻击发生的必要条件。

✅ Step 1: 用户完成正常登录

用户登录银行或社交平台后，浏览器会保存登录凭证（Cookie）。此后访问该网站时，浏览器会自动携带Cookie，服务器据此识别“合法用户身份”，默认信任该请求。

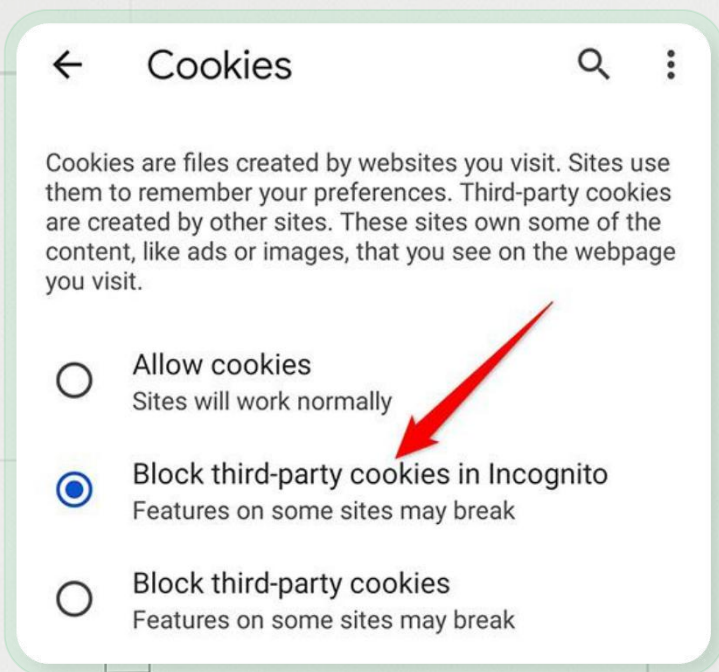
⚠ Step 2: 诱导访问伪造请求

攻击者构造恶意链接或页面，诱导用户在未退出登录时点击。浏览器会自动携带有效Cookie向目标网站发送伪造请求，服务器无法区分请求来源，误判为用户主动操作并执行。

💡 核心认知：攻击者“看不见”Cookie，只“利用”浏览器机制

攻击者无需获取Cookie的具体内容，而是利用浏览器“自动携带Cookie发送请求”的特性。这意味着即使你的密码未泄露，只要保持登录状态并访问恶意页面，就可能中招。

补充小知识：什么是Cookie?



Cookie 是网站存储在浏览器中的一小段文本数据，如同网站颁发的“数字门票”，是实现用户身份识别和个性化体验的核心技术。



记住登录状态

无需重复输入账号密码，关闭网页再次访问时仍保持登录状态，极大提升浏览效率。



保存用户偏好

记录网站主题、语言选择、布局习惯等个性化设置，为用户打造专属的浏览体验。



安全风险提示：在 CSRF（跨站请求伪造）攻击中，Cookie 可能成为被恶意利用的“通行证”。攻击者可利用用户的登录状态，在用户不知情的情况下发起非授权操作。

02 CSRF攻击真实场景举例



01 身份冒用：资金转移

攻击者构造虚假转账链接，诱导已登录网银的用户点击。由于请求携带合法Cookie凭证，银行系统误判为用户本人操作，直接执行资金划转，用户全程毫无察觉。



02 权限篡改：账号接管

诱导用户在登录管理后台或社交账号时访问恶意页面，暗中修改绑定的手机号、安全邮箱或权限配置。攻击者借此重置密码，完全掌控账号，窃取敏感数据。



03 恶意传播：声誉破坏

利用用户的登录状态，在论坛、社交平台自动发布违规内容、广告或钓鱼链接。不仅损害用户个人信誉，还可能引发平台封号、企业信息泄露等连锁反应。

核心洞察： CSRF攻击本质是“借刀杀人”，它不窃取用户密码，而是利用用户的信任身份发起未授权请求。因此，单纯的密码复杂度提升无法有效防御，必须通过Token验证、同源检测等技术手段阻断攻击。

场景一：银行/支付网站转账

1. 汇款单位信息

* 汇款单位:

张三

交易快照

* 汇款账号:

1234567890

可用余额查询

2. 收款单位信息及资金

* 收款单位:

李四

收款人名册

* 收款账号:

0987654321

* 收汇款至个人账户:

是

否

* 收款银行/行别:

直接参与商行

调整限额

* 收款金额:

1,000,000.00

一百万元整

* 汇款方式:

☒ 加急 记为默认

☐ 普通

☐ 跨行快汇

* 收款银行/行别:

浙江

杭州

中国工商银行股份有限公司湖州XX支行

* 收款银行行号

102221009994

3. 汇款用途及备注

* 汇款用途:

货款支付

* 备注:

订单编号: 1234567

攻击者利用浏览器自动携带的Cookie，
在用户无感知下伪造转账请求。



01 登录留痕

用户登录网银后未主动退出，浏览器Cookie中保存了有效的登录凭证，账户处于“免密”授权状态。



02 恶意诱导

攻击者在社交平台发布“领红包”、“抽奖”等极具诱惑的链接，诱导用户在登录网银的浏览器中点击访问。



03 静默请求

恶意页面在后台悄悄发起转账API请求，浏览器因同源策略自动将有效的登录Cookie附加到请求中。



04 资金被盗

银行服务器验证Cookie有效，判定为用户本人操作，直接执行转账指令，资金瞬间被转移至攻击者账户。



隐蔽性极强：整个攻击过程在后台静默执行，用户界面无弹窗、无确认提示，资金在“不知不觉”中被转走，事后才发现异常。

场景二：社交平台账号篡改



// (

核心原理解析

利用用户在浏览器中未退出的登录状态（Cookie凭证），恶意网站在用户不知情的情况下发送伪造的HTTP请求。由于请求携带了合法的身份凭证，服务器会误认为是用户本人操作，从而执行修改资料、发布内容等高危指令。

○ //

{/

攻击链路：隐形的“越权”操作

1. 留存登录态：用户登录社交账号后未退出，浏览器保留Cookie。
2. 触发恶意请求：访问钓鱼网站，页面自动发送篡改信息的请求。
3. 服务器执行：验证Cookie有效，直接执行操作，用户毫不知情。

□ ; (

诱导陷阱：直击人性弱点

攻击者利用极具诱惑力的文案诱导点击，如“性感私密照”、“明星黑料”、“限时福利免费领”等。

高危条件：登录态未清除 + 点击恶意链接 + 平台缺乏CSRF防护。一旦触发，账号控制权瞬间旁落。

03 CSRF漏洞会带来哪些危害



账号权限被恶意接管

攻击者冒用用户身份修改密码、绑定陌生手机号，完全掌控账号控制权，窃取个人隐私信息，甚至利用账号进行违法活动。



财产安全遭受直接威胁

在金融平台场景下，攻击者可诱导用户触发转账、消费、贷款等高危操作，绕过支付验证，直接造成用户资金流失与财产损失。



企业数据与业务受损

若管理员账号中招，攻击者可篡改后台数据、删除关键业务记录，甚至泄露企业核心商业机密，导致运营瘫痪与品牌声誉受损。

核心警示：CSRF漏洞利用“用户已登录”的信任状态发起攻击，其危害隐蔽性极强，且往往直接指向用户财产与平台核心安全，是Web安全防御中必须重点防范的风险点。

CSRF漏洞的危害



对普通用户的直接威胁



财产安全直接受损

攻击者伪造转账、消费或充值请求，在用户不知情下转移资金，造成直接且不可逆的经济损失。



账号控制权被窃取

恶意修改个人资料、绑定手机号，甚至删除数据或发布违规内容，导致账号完全失控。



敏感隐私信息泄露

非法调用查询接口，获取身份证、手机号、消费记录等隐私数据，造成严重的个人信息泄露风险。



对企业与平台的深远冲击



品牌信任与声誉崩塌

用户因安全漏洞遭受损失后，将迅速失去对平台的信任，导致用户大量流失，市场口碑难以挽回。



法律追责与经济赔偿

因平台安全防护不力导致用户受损，企业将面临监管部门的处罚、用户的法律诉讼及高额的经济赔偿。



业务秩序失控与瘫痪

大量账号被篡改产生海量垃圾数据，干扰平台正常运营，甚至引发服务中断，造成巨大的运维成本。

04 如何检测CSRF漏洞



请求伪造验证

核心检测请求是否仅依赖 Cookie 进行身份认证，而未部署 CSRF Token、Referer 校验等防御机制。若请求可被跨域构造并成功执行，则判定存在漏洞风险。



自动化工具扫描



借助 Burp Suite、OWASP ZAP 等专业工具，自动抓取业务请求并分析参数特征。快速识别请求中是否缺失 Token、Token 是否可预测或校验逻辑失效等问题。



人工渗透验证

模拟攻击者视角，手动构造跨站请求（如恶意链接、自动提交表单），在不同权限场景下测试请求是否被成功执行，验证系统对请求来源的校验是否有效。

检测核心逻辑：判断漏洞的关键在于验证系统是否对请求的“来源合法性”进行了严格校验，以及是否为每个敏感操作生成了唯一、不可预测且绑定用户会话的 CSRF Token，并在服务端正确验证其有效性。

CSRF漏洞检测方法

01 手动检测 · 人工观察

核心思路：模拟攻击者构造跨站请求，验证服务器是否在未校验身份的情况下直接执行操作。

关键步骤：执行敏感操作（如修改密码），检查请求中是否包含随机的CSRF Token。若无Token且仅依赖Cookie校验，则存在高危风险。

02 工具辅助 · Burp Suite



高效检测：利用专业安全工具进行自动化分析。

操作流程：拦截请求数据包 → 分析参数构成 → 自动生成CSRF测试页面(POC) → 在登录状态下验证是否可执行伪造操作。



法律红线警示：渗透测试与漏洞检测必须在拥有合法授权的前提下进行。未经授权对他人网络系统进行扫描、攻击或测试，属于违法行为，将面临法律追责。请恪守网络安全法律法规。

05 CSRF漏洞的防范手段



前端防护：阻断恶意触发

01 同源策略校验： 利用浏览器同源策略，严格限制跨域请求的发送，从源头阻断非法调用。

02 敏感操作二次验证： 在执行转账、改密等高危操作时，强制要求输入验证码或进行短信验证。

03 规范请求方法： 严禁使用GET请求处理数据修改类操作，防止攻击者通过URL诱导触发。



后端防护：核心验证机制

01 CSRF Token 校验： 为每个用户会话生成唯一随机Token，嵌入表单或请求头，服务器端严格校验。

02 SameSite Cookie 属性： 设置Cookie的SameSite为Strict或Lax，限制第三方网站携带Cookie发起请求。

03 请求头来源验证： 校验HTTP请求的Referer和Origin字段，确认请求来自合法的受信任域名。

防护建议： 单一手段难以完全杜绝风险，建议采用“Token验证+同源检测+敏感操作二次确认”的组合策略，构建多层防御体系，最大化降低攻击风险。

前端辅助防护（辅助手段）



01. 验证码与短信二次校验

针对转账、密码修改等高风险操作，强制触发验证码校验。攻击者无法窃取用户的实时验证码，可直接阻断伪造请求的有效性，是防范身份冒用的核心辅助手段。

⚠ 局限性：会增加用户操作步骤，影响体验，仅建议在高危操作场景下启用。



02. 校验请求来源 Referer 头

后端校验请求头中的 Referer 字段，仅允许来自本站域名的请求访问，直接拦截外部非法域名发起的跨站请求，从源头阻断非法访问路径。

⚠ 局限性：Referer 可被浏览器插件或代理工具篡改或丢弃，无法作为唯一的安全校验依据。

后端核心防护

```
</div>
</div>
</div>
<script>
  // 登录验证 可删除
  function loginCheck () {
    // 获取id为用户的value值
    var user = document.getElementById('user').value
    // 获取id为password的value值
    var password = document.getElementById('password').value

    // 判断账号是否输入
    if(!user) {
      alert("账号不能为空")
      // 没输入就打断
      return false
    }
    // 判断密码是否输入
    if(!password) {
      alert("密码不能为空")
      // 没输入就打断
    }
  }
}
```

图示：代码层实现的 Token 校验逻辑，确保每一次请求的合法性与唯一性。

CSRF-Token：业界最主流的防御方案

- 

01 生成唯一Token

用户打开表单时，服务器生成随机、一次性的加密Token，确保不可预测。
- 

02 双向安全存储

Token存入服务端Session，同时嵌入表单隐藏字段随页面下发给用户。
- 

03 请求必须携带

用户提交操作时，表单数据必须与Token一并提交，二者缺一不可。
- 

04 严格比对验证

后端校验请求Token与Session中存储的是否一致，不匹配则直接拦截。
- 

为什么有效？——攻击者无法窃取Token

恶意页面无法获取用户Session中的随机Token，伪造的请求因缺少有效Token而被服务器直接拒绝，从源头阻断跨站请求伪造攻击。

后端核心防护（续）

01 设置 Cookie 的 SameSite 属性

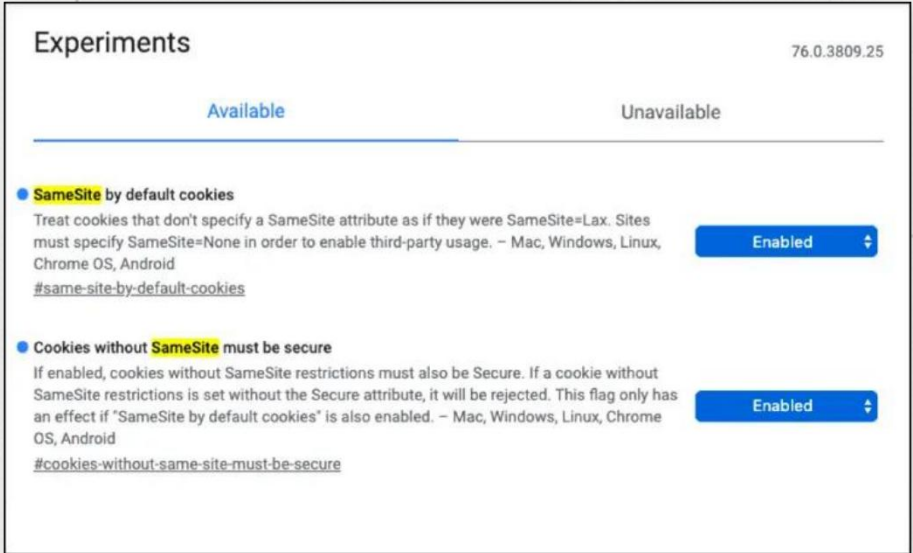
通过配置 Cookie 的 SameSite 属性，明确告知浏览器跨站请求时是否携带 Cookie，从浏览器层面阻断 CSRF 攻击的核心条件，是现代 Web 安全的基础防护手段。

- Strict 严格模式：完全禁止跨站请求携带 Cookie，防护强度最高，适用于登录态、支付等极高敏感场景。
- Lax 宽松模式（默认）：仅允许安全的跨站导航（如链接跳转）携带 Cookie，在保障安全的同时，不影响正常的用户交互体验。

⚠️ 关键提示：该属性依赖浏览器支持，无法完全替代 CSRF-Token 校验。建议将 SameSite 属性、Token 校验与业务层面的二次确认结合使用，构建多层纵深防御体系。

02 高危操作强制二次确认

针对转账、改绑手机、重置密码等核心操作，强制叠加独立密码或短信验证码验证，作为账户安全的最后一道防线。



Chrome 浏览器默认开启 SameSite=Lax 策略配置界面

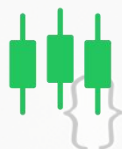
🛡️ 防护共识：技术层面的规则限制（Token/SameSite）必须配合业务层面的身份验证，才能实现真正的安全闭环。

06 总结与展望



回顾与沉淀

系统复盘年度安全建设成效，完成核心资产梳理与漏洞治理，建立标准化的安全管理制度与应急响应流程，为业务稳定运行筑牢坚实的安全根基。



挑战与洞察

AI技术加速催生新型网络攻击手段，攻击面持续扩大，防御边界日益模糊。需洞察威胁演变趋势，构建自适应、智能化的防御体系，化挑战为提升安全能力的契机。



展望与布局

未来将深化零信任架构落地，推进安全运营自动化与智能化，融合AI赋能威胁检测与响应，构建主动防御、持续迭代的安全生态，护航数字化业务高速发展。

安全不止于防御，更是业务创新的护航者。以技术为盾，以管理为纲，共筑可信、可控、可演进的数字安全未来。

要点回顾



01 核心本质：身份冒用

攻击者并非窃取密码，而是利用用户已登录的合法身份，冒充用户发起恶意请求，本质是“借刀杀人”式的身份冒用。



02 攻击三要素：缺一不可

① 用户在目标网站保持登录状态；② 访问攻击者的恶意诱导页面；③ 目标网站未部署有效的CSRF防护机制。



03 主要危害：双重打击

对用户：资金被盗、账号被篡改、隐私泄露；对企业：面临数据合规风险、品牌声誉受损及法律追责。



04 检测方法：查验Token

通过抓包分析关键操作请求，若未携带随机生成的CSRF-Token，或Token可被预测、复用，则存在被攻击风险。



05 防御重点：多重防护体系

核心是后端严格校验CSRF-Token；辅助手段包括设置Cookie SameSite属性、高危操作增加验证码、校验Referer请求头。



开发者安全贴士

规范编码：敏感接口切勿仅依赖Cookie，必须结合Token校验。

协议认知：理解Cookie作用域与Token防伪造机制，分析HTTP请求。

合规底线：安全测试务必在授权环境进行，严禁触碰法律红线。